The Medium

1. Things

The simplest things we can write are names that stand only for themselves, like `rock`. We say what kind of thing they are by listing every thing of that kind:

```
Hand = rock | paper | scissors
Bool = false | true
```

`|` reads "or". Each line **declares** a **type**: a kind of thing. A `Hand` is `rock`, `paper` or `scissors`, and nothing else. These names are **constructors**. Types are written with a capital letter; constructors, and every other name, without. Each type is declared once, and each constructor belongs to one type.

Something we write to be worked out is an **expression**. A constructor on its own is the simplest one: there is nothing left to work out in it. An expression like that is a **value**.

As we go, we keep a **map**: a list of every way of writing things we have met so far, with letters for the parts that can change. `T` stands for any type's name, `c` for any constructor, `e` for any expression, `v` for any value. `::=` reads "is one of", and `| …` reads "and more like it, as many as needed". Right now an expression and a value are both just a constructor; they will grow apart. Under these go the rules, each tagged with what it is about.

MAP

```
T = c | …          a type, with its constructors
e ::= c            a constructor
v ::= c            a constructor

names   each type is declared once, and each constructor belongs to one type
```

2. Functions

A **function** turns things into other things. We write it as **cases**: for each thing it can be given, what it gives back.

```
better = (rock) ↦ paper | (paper) ↦ scissors | (scissors) ↦ rock
```

`↦` reads "gives": rock gives paper, paper gives scissors, scissors gives rock. Each hand gives the hand that beats it. In a case, what is in parentheses is its **pattern**: what the case is for. What comes after `↦` is its **body**: what it gives back. `better = …` **defines** the name `better`: from now on, `better` stands for this function.

To use a function, we **apply** it to an **input**, written after it in parentheses. Applying takes one **step**: the application **becomes** the body of the first case whose pattern the input **matches**. A constructor matches only itself.

```
better(rock)
    becomes   paper
```

Nothing is left to work out in `paper`: it is a value, so it is the **result**. Taking steps like this, one after another, until nothing is left to work out, is **carrying out** the expression. A computer does it, and so can we, on paper.

One more function, which turns each `Bool` into the other:

```
not = (false) ↦ true | (true) ↦ false
```

A function can take several inputs. Each case then has a pattern for each input, and the inputs match a case when each matches its own pattern. `and` asks whether both of two `Bool`s are `true`:

```
and = (true, true) ↦ true
    | (true, false) ↦ false
    | (false, true) ↦ false
    | (false, false) ↦ false

and(true, false)
    becomes   false
```

An input can itself be an application. It is worked out first, and then it goes in. Several inputs are worked out one at a time, from left to right:

```
and(not(false), not(true))
    becomes   and(true, not(true))
    becomes   and(true, false)
    becomes   false
```

So there is never more than one next step. "Comes to" means "becomes, in one step or more": `and(not(false), not(true))` comes to `false`.

A function can also take no inputs, and is applied to none:

```
later = () ↦ better(rock)

later()
    becomes   better(rock)
    becomes   paper
```

Nothing inside a function is worked out until it is applied, and then only the body of the case that is used. So `later()` is `better(rock)`, kept to be worked out later. Section 12 uses this.

The cases of one function all have the same number of patterns. That is how many inputs it takes, and it is always given that many: `and(true)` is not an expression. Only a function can be applied: `rock(paper)` is not an expression either. A definition can use the names defined before it, as `later` uses `better`.

What if a function is given something it has no case for?

```
better(true)          stuck: no case of better matches true
```

Something is left to work out, but no step applies. Carrying out is **stuck**. So carrying out ends in one of two ways: in a value, which is the result, or stuck.

What we write is declarations and definitions, then one expression to carry out. All of it together is a **program**.

On the map, `n` stands for any defined name and `p` for any pattern. `p, …` means any number of patterns, even none, and likewise `e, …` and `v, ….` A new `e ::=` line adds one more kind of expression to those before it:

```
n = (p, …) ↦ e | …     a definition: n stands for a function, with its cases
p ::= c                a constructor: matches only itself
e ::= n(e, …)          an application

n(v, …)
    becomes  the body of the first case in n's definition
             whose patterns match the inputs v, …

names   each defined name is defined once
names   a definition can use the names defined before it
names   the cases of one function all have the same number of patterns:
        how many inputs it takes; it is always applied to that many
order   in n(e, …), the inputs are worked out from left to right;
        then n(v, …) takes its step
order   nothing happens inside a function until it is applied
ends    in a value: the result
ends    stuck: something is left to work out, but no step applies
```
MAP

3. Placeholders

`and` says the same thing more than once. When the first input is `true`, the result is the second input, whatever it is. When it is `false`, the result is `false`. To say that, a pattern can be a **placeholder**: a name that matches anything, and stands, in the body, for what it matched.

```
and = (true, b) ↦ b | (false, b) ↦ false

and(true, true)
    becomes   true
```

`b` matched `true`, and the body `b` gives it back.

This `and` replaces the one in section 2. A program defines each name once; when this book writes a definition or a declaration again, the new one is the one we use from then on.

Cases are tried in order, so a last case with a placeholder means "otherwise":

```
beats = (rock, scissors) ↦ true
      | (scissors, paper) ↦ true
      | (paper, rock) ↦ true
      | (a, b) ↦ false

beats(paper, rock)      becomes   true
beats(rock, paper)      becomes   false
```

Rock beats scissors, scissors beat paper, paper beats rock. Otherwise, the first hand doesn't win. Without placeholders, this would take nine cases.

A body can use a placeholder anywhere, even deep inside applications:

```
beaten = (a) ↦ better(better(a))

beaten(rock)
    becomes   better(better(rock))
    becomes   better(paper)
    becomes   scissors
```

`beaten` gives the hand that a hand beats: what beats what beats it. Rock beats scissors.

In a pattern, a name is a constructor if it was declared as one, and otherwise a placeholder. The placeholders of one case are all different: in `(a, a)`, we couldn't tell which input `a` is. And none has the

name of a defined name: if a placeholder were called `better`, we couldn't tell whether `better` in the body meant the function or what the placeholder matched. A placeholder means something only in the body of its own case. So in a body, every name means one thing.

On the map, `x` stands for any placeholder. A line marked widened replaces the earlier line it grows from, and allows more:

```
p ::= x                a placeholder: matches anything                          MAP
e ::= x                a placeholder

n(v, …)
    becomes  the body of the first case in n's definition
             whose patterns match the inputs v, …,
             with what each placeholder matched in its place              widened

names   in a pattern, a name is a constructor if it was declared as one,
        and otherwise a placeholder
names   the placeholders of one case are all different, and none has the name
        of a defined name
names   a placeholder means something only in the body of its own case
```

4. Data

A round of the game is two hands, one for each player. A constructor can **hold** things, its **contents**, of the types written after it:

```
Round = round(Hand, Hand)
```

`Round`, with a capital, is the type; `round` is its constructor. A constructor that holds things is always written with them, as many as it holds:

```
round(rock, paper)
```

This is not an application. `round` is not a function, and nothing happens: `round(rock, paper)` is a value, a round where the first player showed rock and the second paper. Values built from constructors are **data**. Contents are worked out before they go in, from left to right, like inputs:

```
round(better(rock), rock)
    becomes   round(paper, rock)
```

To use data, we take it apart with a pattern: the constructor, with patterns for its contents.

```
won = (round(a, b)) ↦ beats(a, b)

won(round(paper, rock))
    becomes   beats(paper, rock)
    becomes   true
```

`round(a, b)` matches any round, and gives `a` the first hand and `b` the second. The contents of a pattern are patterns, so a pattern can look more than one layer in:

```
draw = (round(rock, rock)) ↦ true
     | (round(paper, paper)) ↦ true
     | (round(scissors, scissors)) ↦ true
     | (other) ↦ false

draw(round(paper, rock))
    becomes   false
```

`round(paper, rock)` doesn't match `round(paper, paper)`, since `rock` doesn't match `paper`. Only the last case matches.

A declaration can use the types declared before it, as `Round` uses `Hand`. A constructor that holds nothing is written without parentheses: `rock`, never `rock()`.

On the map, `A` stands for any type, and `d` for a constructor as it is declared: just its name, or its name with the types it holds.

```
T = d | …              a type, with its constructors                    widened        MAP
d ::= c                a constructor holding nothing
      c(A, …)          a constructor holding contents of types A, …
A ::= T                a type, by its name
e ::= c(e, …)          data: a constructor with its contents
v ::= c(v, …)          data: a constructor holding values
p ::= c(p, …)          matches data built by c, with contents matching p, …

names   a constructor is written with as many contents as it holds, in an expression
        or a pattern; one that holds nothing, without parentheses
names   a declaration can use the types declared before it
order   in c(e, …), the contents are worked out from left to right
```

5. Repeating

Some kinds of thing have no end. A constructor can hold things of its own type:

```
Nat = zero | succ(Nat)
```

`succ` reads "one more than". `zero`, `succ(zero)`, `succ(succ(zero))` are zero, one, two, and so on, without end.

A pattern like `succ(a)` looks one layer in. To work through a number of any size, a function has to go around once for each layer, so it has to use itself. So far, a definition could use only the names defined before it. From here on, it can use its own:

```
add = (zero, b) ↦ b | (succ(a), b) ↦ succ(add(a, b))
```

Zero plus `b` is `b`. One more than `a`, plus `b`, is one more than `a` plus `b`.

```
add(succ(succ(zero)), succ(zero))
    becomes   succ(add(succ(zero), succ(zero)))
    becomes   succ(succ(add(zero, succ(zero))))
    becomes   succ(succ(succ(zero)))
```

Each step takes a layer off the first input and puts it outside, until `zero` ends it. Two and one make three.

A definition can also use a name defined after it, so two definitions can use each other:

```
even = (zero) ↦ true | (succ(a)) ↦ odd(a)
odd = (zero) ↦ false | (succ(a)) ↦ even(a)

even(succ(succ(zero)))
    comes to  true
```

Declarations work the same way: a type can hold any type, its own included, even one declared after it. `Round` could just as well be declared before `Hand`.

Nothing makes a function use itself on something smaller. So carrying out can go on forever:

```
loop = (a) ↦ loop(a)

loop(rock)
    becomes   loop(rock), and so on, forever
```

Repeating needs no new form: only a name used inside its own definition.

```
MAP
names   a definition can use any defined name, its own included,
        even one defined after it                                  widened
names   a declaration can use any type, its own included,
        even one declared after it                                 widened
ends    never: carrying out goes on forever
```

6. Types left open

A list of hands is either empty, or a hand followed by a list of hands. A list of rounds is the same, with rounds instead of hands. Only the type of the items changes. So a type can leave that open, with **placeholders for types** after its name:

```
List(Item) = empty | cons(Item, List(Item))
Pair(One, Two) = pair(One, Two)
```

A `List(Hand)` is `empty`, or `cons` holding a hand and a `List(Hand)`: `Item` is filled with `Hand`. `cons` reads "followed by", so `cons(rock, cons(paper, empty))` is rock, then paper. A `Pair(Bool, Nat)` holds a `Bool` and a `Nat`, as `pair(true, zero)` does. Types nest: a `List(List(Hand))` is a list of lists of hands.

`Item` means something only inside `List`'s own declaration. The placeholders of one declaration have different names, like `One` and `Two`, and none is named like a declared type. `List` is always filled with one type, and `Pair` with two.

Functions need nothing like this. A function looks only at how a value is built, so one that works through a list works through a list of anything:

```
size = (empty) ↦ zero | (cons(a, rest)) ↦ succ(size(rest))

size(cons(rock, cons(paper, empty)))
    comes to  succ(succ(zero))
size(cons(true, empty))
    comes to  succ(zero)
```

On the map, `X` stands for any placeholder for a type, like `Item`:

```
MAP
T(X, …) = d | …        a type that takes types; T = d | … takes none        widened
A ::= T(A, …)          a type, filled with types; T alone if it takes none  widened
      X                a placeholder for a type

names   in a declaration, every name in a type is a declared type,
        or one of the declaration's own placeholders
names   the placeholders of one declaration are all different,
        and none has the name of a declared type
names   a type is filled with as many types as it takes
```

7. Functions as values

So far a defined name was always applied. It can also be written on its own. Then it stands for its function, and it is a value: nothing in it is worked out until it is applied. So a function can be given to a

function:

```
twice = (f, a) ↦ f(f(a))

twice(better, rock)
    becomes   better(better(rock))
    comes to  scissors
```

What is applied here is a placeholder, `f`. So what is applied can now be any expression. It is worked out first, then the inputs, and then the application takes its step. `each` gives a function every item of a list:

```
each = (f, empty) ↦ empty | (f, cons(a, rest)) ↦ cons(f(a), each(f, rest))

each(better, cons(rock, cons(paper, empty)))
    comes to  cons(paper, cons(scissors, empty))
```

A function can also be **written out**, without a name, wherever an expression can go. It is applied the same way:

```
((rock) ↦ paper | (paper) ↦ scissors | (scissors) ↦ rock)(rock)
    becomes   paper
```

The parentheses around the function keep it in one piece. Without them, the `(rock)` at the end would be read as part of the last body, since a body reaches as far as it can. Parentheses around a whole expression, like these, only group it; they change nothing else. And this is the same function as `better`: all a definition adds is a name.

Written out inside a body, a function can use the placeholders around it. That is how a function gives back a function made from its input:

```
against = (a) ↦ (b) ↦ beats(b, a)

against(rock)
    becomes   (b) ↦ beats(b, rock)
against(rock)(paper)
    comes to  true
```

`against(rock)` is a test: does a hand beat rock? The input went wherever `a` was, even inside the inner function, where nothing is worked out yet.

A function can take a function and give back a function:

```
twofold = (f) ↦ (a) ↦ f(f(a))

twofold(better)
    becomes   (a) ↦ better(better(a))
```

That is `beaten`, made without a definition of its own. If an inner placeholder has the same name as an outer one, the name means the nearer one:

```
((a) ↦ (a) ↦ a)(rock)(paper)
    becomes   ((a) ↦ a)(paper)
    becomes   paper
```

In the first step, `rock` doesn't go into the inner function: there, `a` means the inner function's own placeholder.

Data can hold functions too. A player plays by a rule: given the hand the other player showed last, it gives the hand to show next. The type of such rules is a **function type**:

```
Player = player((Hand) → Hand)
```

`(Hand) → Hand` reads "from `Hand` to `Hand`".

```
next = (player(f), a) ↦ f(a)

next(player(better), rock)
    becomes   better(rock)
    becomes   paper
```

No pattern looks inside a function. A function can only be applied.

Every value is data or a function, and either can go wherever a value goes:

`(Bool) → Bool`	takes data, gives data	`not`
`(Hand) → (Hand) → Bool`	takes data, gives a function	`against`
`((Hand) → Hand, Hand) → Hand`	takes a function, gives data	`twice`
`((Hand) → Hand) → (Hand) → Hand`	takes a function, gives a function	`twofold`
`Round`	data holding data	`round(rock, paper)`
`Player`	data holding a function	`player(better)`

`(Hand) → (Hand) → Bool` reads "from `Hand` to `(Hand) → Bool`": it takes a hand, and gives back a function.

Now that anything can be applied, carrying out can get stuck in two more ways:

```
twice(rock, rock)       becomes rock(rock(rock)): stuck, rock is not a function
twice(beats, rock)      becomes beats(beats(rock)): stuck, beats takes two inputs
```

We can't write `rock(rock)` or `beats(rock)` ourselves: the map rules them out, since we can see the name. Through a placeholder, we can't see it coming. Only carrying out gets there, and there it is stuck.

On the map, `B`, like `A`, stands for any type:

MAP

```
e ::= n                    a defined name
      (p, …) ↦ e | …       a function written out
      e(e, …)              an application                    widened from n(e, …)
v ::= n                    a defined name
      (p, …) ↦ e | …       a function written out
A ::= (A, …) → B           a function type

((p, …) ↦ e | …)(v, …)
    becomes  the body of the first case whose patterns match the inputs v, …,
             with what each placeholder matched in its place

names   a placeholder means something in the body of its own case,
        functions written inside it included                              widened
names   in an expression, a name means the nearest placeholder of that name;
        with none, it is a constructor or a defined name
names   a defined name written right before inputs is given
        as many as it takes                                               widened
order   in e(e, …), first what is applied, until it is a value;
        then the inputs, from left to right                               widened
ends    stuck also when what is applied is not a function,
        or takes a different number of inputs
writing parentheses around an expression only group it
writing a body reaches as far as it can
```

8. What a type means

A type says which values are of a kind. `rock` is a `Hand`: it is one of `Hand`'s constructors. `round(rock, paper)` is a `Round`: it is `round` holding two hands. `cons(rock, empty)` is a `List(Hand)`: it is `cons` holding a hand and a `List(Hand)`. In general, a value is of type `T` when it is built by one of `T`'s constructors, holding values of the types declared for its contents.

`round(true, false)` can still be written and carried out. It is data, just not a `Round`. A type says what a kind is; it doesn't stop anything from being written.

`better` and `beaten` are `(Hand) → Hand`: given any hand, they never get stuck, and they end in a hand. `(rock) ↦ paper` is not: given `paper`, it gets stuck. In general, a value is of type `(A, …) → B` when, applied to values of types `A, …`, it never gets stuck, and if it ends, it ends in a value of type `B`.

A type that holds itself is read over and over: `succ(succ(zero))` is a `Nat` because `succ(zero)` is, and `succ(zero)` is because `zero` is. A type can also hold itself in what a function gives back:

```
Game = over(Bool) | yourTurn((Hand) → Game)
```

A game is over, saying whether you won, or it is your turn: given your hand, it gives the rest of the game. `yourTurn((a) ↦ over(beats(a, rock)))` is one round against rock. A game can also go on forever:

```
endless = (a) ↦ yourTurn(endless)
```

`yourTurn(endless)` is a `Game`: following it never ends, but it never finds anything that isn't a `Game`. So a value is of a type that holds itself when following the definition, as far as it can be followed, never finds anything that doesn't fit.

This goes in a circle: what counts as a `Game` depends on what counts as a `Game`. For `Game`, the circle settles: the more values count as `Games`, the more functions give back `Games`, so the two grow together. But a type can't hold itself in what a function takes. In `Test = test((Test) → Bool)`, the more values count as `Tests`, the more inputs a `(Test) → Bool` must take without getting stuck, so the fewer functions count, and the fewer values are `Tests`. Counting more makes fewer: the circle never settles, and the definition says nothing.

Nothing in carrying out looks at types. A type is for us: it says what kind of value we mean. Whether a value is of that kind is something we can check, but checking it is not a step.

MAP

```
types   a value is of type T(A, …) when it is built by one of T's constructors,
        holding values of the types declared for its contents,
        with A, … in place of T's placeholders
types   a value is of type (A, …) → B when, applied to values of types A, …,
        it never gets stuck, and if it ends, it ends in a value of type B
types   a type that holds itself: a value is of it when following the definition,
        as far as it can be followed, never finds anything that does not fit
types   a type can hold itself in its contents, and in what a function gives back,
        but not in what a function takes
```

9. A score that changes

Everything so far is **saying**: an expression and what it comes to say the same thing, the second more directly. Nothing changes. Keeping score is **doing**: something changes as we go.

As they play, the players keep score. The score is a **memory**: something kept, and changed as we go. With what we have, a memory has to be handed along as a value. Take a function that asks whether the

first player won both of two rounds:

```
both = (first, second) ↦ and(won(first), won(second))
```

To keep the score as well, each round has to give back two things: whether it was won, and the new score.

```
more = (true, total) ↦ succ(total) | (false, total) ↦ total
play = (this, total) ↦ pair(won(this), more(won(this), total))
```

`more` adds one to the score if the round was won. `play` gives back whether it was won, paired with the new score. To use that, we take the pair apart. A function written out and applied at once does it: its pattern gives names to the parts.

```
((pair(a, m)) ↦ a)(pair(true, zero))
    becomes   true
```

Now `both` has to take the score, and hand it through each round. Play the first round with `total`, and call what comes back `a` and `m`. Play the second with `m`, and call what comes back `b` and `t`. Give back `and(a, b)`, with the score `t`:

```
both = (first, second, total) ↦
    ((pair(a, m)) ↦
        ((pair(b, t)) ↦ pair(and(a, b), t))(play(second, m)))
    (play(first, total))

both(round(paper, rock), round(rock, paper), zero)
    comes to  pair(false, succ(zero))
```

`both` doesn't care about the score. It only hands it along. And so must every function that uses `both`, and every function that uses those. One small idea touches everything on the way.

We want to mention the score only where it is used. No combination of what we have does that: the score has to pass through every function on the way. So doing gets forms of its own: requests and handlers.

10. Requests and handlers

Take a simpler case of the same problem first. To know whether a hand wins, a function needs the hand the opponent plays. Instead of being handed it, the function can ask for it:

```
effect Opponent = opponent : () → Hand
```

This declares an **effect**, `Opponent`: a kind of asking. Its one **request**, `opponent`, asks for the opponent's hand. `opponent : () → Hand` reads "`opponent` takes nothing, and is answered with a `Hand`". Like a type, each effect is declared once, and each request belongs to one effect. A request is written like an application, with as many inputs as it is declared with:

```
beatsOpponent = (a) ↦ beats(a, opponent())

beatsOpponent(rock)
    becomes   beats(rock, opponent())
```

And there it waits. A request is not a value, and takes no step of its own: it waits to be answered.

A **handler** answers it. It goes around the expression that asks:

```
handle beatsOpponent(rock) with
      (outcome) ↦ outcome
    | (opponent(), k) ↦ k(scissors)
```

A handler has cases, like a function. The first is for the value the expression inside comes to: here, it gives that value back. The second is for the request `opponent()`. It answers `scissors`, by giving it to `k`. In a handler's case, `opponent` means the request; no placeholder is ever named like a request.

`k` is **the rest**: what was left to do when the request was made, as a function. `beats(rock, opponent())` is waiting for a hand to put where `opponent()` is. Given one, the rest puts it there and goes on, inside the same handler. Here and from now on, `with …` stands for that handler's cases, written again:

```
(hand) ↦ handle beats(rock, hand) with …
```

When the request is the next thing to work out, the handler takes a step: it becomes the body of its case for the request, with the rest in place of `k`.

```
handle beatsOpponent(rock) with …
    becomes   handle beats(rock, opponent()) with …
    becomes   ((hand) ↦ handle beats(rock, hand) with …)(scissors)
    becomes   handle beats(rock, scissors) with …
    becomes   handle true with …
    becomes   true
```

In the last step, the expression inside is a value, so the handler becomes the body of its case for values.

`beatsOpponent` never says where the opponent's hand comes from. The handler does. A handler that answered `paper` would make the same `beatsOpponent(rock)` come to `false`, without changing `beatsOpponent`.

A request that a handler has no case for passes it by, to a handler further out:

```
handle (handle beatsOpponent(rock) with (outcome) ↦ not(outcome)) with
      (outcome) ↦ outcome
    | (opponent(), k) ↦ k(scissors)

    comes to  false
```

The inner handler has no case for `opponent()`, so the outer one answers. The rest goes on inside both handlers, so the inner one still gets `true`, and turns it into `false`.

With no handler around it to answer, a request is **waiting**, and carrying out ends there: `beatsOpponent(rock)` on its own comes to `beats(rock, opponent())`, and waits. On the map, `F` stands for any effect's name, `r` for any request, and `h` for a handler's case:

```
effect F = r : (A, …) → B | …    an effect, with its requests:                    MAP
                                 what each takes, and what it is answered with
e ::= r(e, …)                    a request
      handle e with h | …        a handler around e
h ::= (p) ↦ e                    for the value the expression inside comes to
      (r(p, …), x) ↦ e           for a request: patterns for its inputs,
                                 and a placeholder for the rest

handle v with h | …
    becomes  the body of the first case for values that v matches,
             with what each placeholder matched in its place
handle e with h | …
    becomes  the body of the first case for r whose patterns match the inputs v, …,
             with what they matched in place, and the rest in place of x,
             when the next thing to work out in e is a request r(v, …),
             and no handler in e around that request has a case for r

rest    the rest is (x) ↦ handle e′ with h | …, where e′ is e with x
        where the request was, and x is a placeholder used nowhere
        in e or in the handler's cases
names   each effect is declared once, and each request belongs to one effect
names   a request is written with as many inputs as it is declared with,
        in an expression or in a handler's case; no placeholder has its name
order   in r(e, …), the inputs are worked out from left to right
order   in handle e with h | …, steps happen in e
ends    waiting: the next thing to work out is a request
        that no handler around it has a case for
ends    stuck: something is left to work out, no step applies,
        and it is not waiting                                          widened
```

11. Never, or several times

The handler above went on once. But it holds the rest as a value, `k`, and can do with it whatever can be done with a function.

It can never go on. That is how errors work. `firstOf` gives the first item of a list. An empty list has none, so there it asks to fail:

```
effect Fail = fail : () → Hand

firstOf = (cons(a, rest)) ↦ a | (empty) ↦ fail()

handle firstOf(empty) with
      (outcome) ↦ outcome
    | (fail(), k) ↦ rock

    comes to  rock
```

The handler never applies `k`, so the rest never happens: its answer, `rock`, stands in for the whole. Without the request, `firstOf(empty)` would be stuck. With it, the handler decides what happens instead.

It can go on several times. This handler tries every hand the opponent could play, and counts the times rock wins:

```
count = (true) ↦ succ(zero) | (false) ↦ zero

handle beatsOpponent(rock) with
      (outcome) ↦ count(outcome)
    | (opponent(), k) ↦ add(k(rock), add(k(paper), k(scissors)))

    comes to  succ(zero)
```

Each `k(…)` goes on from the request with a different hand, and comes through the first case to a number: `k(rock)` and `k(paper)` come to `zero`, and `k(scissors)` to `succ(zero)`. Rock wins once. This is search: one question, every answer tried.

12. Memory

Now the score. An effect can take placeholders for types, as a type can:

```
Unit = unit
effect State(Kept) = get : () → Kept | put : (Kept) → Unit
```

In `State(Nat)`, a memory of a number, `get` asks what the memory holds, and `put` sets it. `put` has nothing to give back, so it is answered with `unit`, which says nothing. To do one thing and then another:

```
then = (u, a) ↦ a
```

Inputs are worked out from left to right, so `then` works out its first input, then its second, and gives back the second. Now recording a round:

```
record = (true) ↦ then(put(succ(get())), true) | (false) ↦ false
```

Given that a round was won, `record` gets the score, puts back one more, and then gives back `true`. Given that it was lost, it just gives back `false`. And `both`, once more:

```
both = (first, second) ↦ and(record(won(first)), record(won(second)))
```

Only `record`, the one place that changes the score, mentions it. Nothing is handed along, and nothing that uses `both` has to change.

Something still has to answer `get` and `put`. Nothing we write ever changes, so the handler can't keep the score in a box and change it. It does what section 9 did: it hands the score along, from each request to the next. But it does it in one place:

```
keep = (task, start) ↦ (handle task() with
                          (outcome) ↦ ((m) ↦ pair(outcome, m))
                        | (get(), k) ↦ ((m) ↦ k(m)(m))
                        | (put(b), k) ↦ ((m) ↦ k(unit)(b)))(start)
```

- `task` is the work to do, kept for later as a function with no inputs. Given directly, it would be worked out before it went in, and make its requests before the handler was around them. Kept for later, it starts at `task()`, inside the handler.
- The handler doesn't give back an answer directly. Every case gives back a function waiting for the memory as it is now, `m`. That is how the memory is handed along. The `(start)` at the end gives it the first one.
- At a value: given `m`, give back the value together with `m`.
- At `get()`: given `m`, answer `m`, and go on with the memory still `m`.
- At `put(b)`: given `m`, answer `unit`, and go on with the memory now `b`.

- The rest, `k`, has the same handler around it, so what it gives back is also waiting for the memory. That is why it is given the memory in turn: `k(m)(m)`, `k(unit)(b)`.

Two rounds, the first won and the second lost, starting from zero. `k` is the rest after the first request, `get()`, and `k′` ("k prime", just another name) the rest after the second, `put(succ(zero))`. Each "comes to" skips the steps up to the next request; the note on the right says where it stopped:

```
keep(() ↦ both(round(paper, rock), round(rock, paper)), zero)
    comes to  (handle both(round(paper, rock), round(rock, paper)) with …)(zero)
    comes to  ((m) ↦ k(m)(m))(zero)                 at get()
    becomes   k(zero)(zero)                         get() is answered zero
    comes to  ((m) ↦ k′(unit)(succ(zero)))(zero)    at put(succ(zero))
    becomes   k′(unit)(succ(zero))                  the memory is now succ(zero)
    comes to  ((m) ↦ pair(false, m))(succ(zero))    both came to false
    becomes   pair(false, succ(zero))
```

Not both were won, and the score is one. The score went along exactly as in section 9, but only the handler and `record` ever mention it.

With requests, order matters. Put first and `get` after, and `get` finds the new score; `get` first, and it finds the old one:

```
keep(() ↦ then(put(succ(zero)), get()), zero)
    comes to  pair(succ(zero), succ(zero))
keep(() ↦ ((s) ↦ then(put(succ(zero)), s))(get()), zero)
    comes to  pair(zero, succ(zero))
```

In the second, `get()` is an input, so it is worked out first, before `put`. `keep` never looks at the memory, so it keeps a `State(Hand)` as well as a `State(Nat)`:

```
keep(() ↦ then(put(paper), get()), rock)
    comes to  pair(paper, paper)
```

```
effect F(X, …) = r : (A, …) → B | …   an effect that takes types;                    MAP
                                      effect F = … takes none          widened
```

13. Going on later

A handler can also keep the rest in data, and go on later. A player can be a task of its own, like `task` in `keep`, which gives its hands one at a time:

```
effect Give = give : (Hand) → Unit

cycle = (a) ↦ then(give(a), cycle(better(a)))
```

`cycle(rock)` gives rock, then paper, then scissors, then rock again, forever. `cycle` doesn't know who takes its hands, or when. A handler can stop it at each hand, and give back the hand together with the rest:

```
Moves = done | gave(Hand, (Unit) → Moves)

moves = (task) ↦ (u) ↦ handle task() with
                           (outcome) ↦ done
                         | (give(a), k) ↦ gave(a, k)
```

`moves(task)` waits for `unit`, just as `k` does, so nothing starts until someone asks, and starting looks just like going on. Given `unit`, it starts the task and stops at the first hand. The rest, `k`, waits for `unit`, the answer to `give`. Given it, it goes on to the next hand, and stops again. With `k`, `k′` and `k″` the rests after each hand:

```
moves(() ↦ cycle(rock))(unit)      comes to  gave(rock, k)
k(unit)                            comes to  gave(paper, k′)
k′(unit)                           comes to  gave(scissors, k″)
```

`cycle` never ends, but it is carried out only as far as someone asks. This is a generator: it gives its items one at a time, each when it is asked for. `Moves`, like `Game`, holds itself in what a function gives back.

So a handler can go on once, never, several times, or later. The same two forms, request and handler, say all four.

14. The outside

A request that no handler in the program answers is waiting, and the computer running the program can answer it. Showing something on a screen, or reading what someone types, is such a request:

```
effect Console = show : (Hand) → Unit | read : () → Hand

game = () ↦ then(show(rock), beats(rock, read()))
```

`game()` waits at `show(rock)`. The computer shows rock on the screen and answers `unit`, which goes where the request was, and carrying out goes on. At `read()` it waits again, for a hand someone types. The outside works like one more handler, around the whole program, that always goes on once. Inside the program, a handler can stand in for the outside, which is how such a program is tested:

```
handle game() with
      (outcome) ↦ outcome
    | (show(a), k) ↦ k(unit)
    | (read(), k) ↦ k(scissors)

    comes to  true
```

Requests of different effects mix. `keep` has no case for `show`, so here `show(rock)` passes it by and goes outside; once the outside answers `unit`, this comes to `pair(false, succ(zero))`:

```
keep(() ↦ then(show(rock), both(round(paper, rock), round(rock, paper))), zero)
```

A person can be a player, whose rule reads the next hand from the keyboard: `player((a) ↦ read())`. That is data, but not a `Player`: applied to a hand, its rule doesn't end in a hand; it waits at `read()`. A function type can also say what its function may ask:

```
Player = player((Hand) → Hand ! {Console})
```

`(Hand) → Hand ! {Console}` reads "from `Hand` to `Hand`, and may ask `Console`". Given a hand, such a rule may stop at `read()`, or at `show` given a hand. Whatever hand each `read()` is answered with, it must not get stuck, and if it ends, it ends in a hand. Now `player((a) ↦ read())` is a `Player`. "May" includes not asking, so `player(better)` still is one. Carrying out is the same as before:

```
next(player((a) ↦ read()), rock)
    comes to  read()
```

and waits there, for the person to choose. An effect that takes types is filled the same way as a type: `record` is a `(Bool) → Bool ! {State(Nat)}`. On the map, `E` stands for the effects a function may ask:

```
A ::= (A, …) → B ! E     a function type, and what it may ask;                MAP
                         ! E left out when it lists none                widened
E ::= {F(A, …), …}       effects, filled; F alone if it takes none

names   an effect is filled with as many types as it takes
types   a value is of type (A, …) → B ! E when, applied to values of types A, …,
        it never gets stuck, leaves no request unanswered but those of the effects
        in E, with inputs of the types they take, and, with each of those answered
        by a value of the type it is answered with, ends, if it ends, in a value
        of type B                                                      widened
ends    when the outside answers a waiting request, its answer goes where
        the request was, and carrying out goes on
```

15. The whole map

`e` an expression, `v` a value, `x` a placeholder, `c` a constructor, `n` a defined name, `p` a pattern, `d` a constructor as declared, `T` a type's name, `F` an effect's name, `r` a request, `h` a handler's case, `A` and `B` types, `X` a placeholder for a type, `E` the effects a function may ask. `e, …` means any number of them, even none. `| …` means more like the one before. "Becomes" is one step; "comes to" is one or more.

WHAT WE WRITE

A program: declarations and definitions, then one expression to carry out.

MAP

```
T(X, …) = d | …                       a type, with its constructors;
                                      T = d | … when it takes no types
d ::= c                               a constructor holding nothing
      c(A, …)                         a constructor holding contents of types A, …
effect F(X, …) = r : (A, …) → B | …   an effect, with its requests;
                                      effect F = … when it takes no types
n = (p, …) ↦ e | …                    a definition: n stands for a function,
                                      with its cases

parentheses around an expression only group it
a body reaches as far as it can
```

EXPRESSIONS, VALUES, PATTERNS, HANDLER CASES

MAP

```
e ::= x                   a placeholder
      c                   a constructor
      c(e, …)             data: a constructor with its contents
      n                   a defined name
      (p, …) ↦ e | …      a function written out
      e(e, …)             an application
      r(e, …)             a request
      handle e with h | … a handler around e

v ::= c                   a constructor
      c(v, …)             data: a constructor holding values
      n                   a defined name
      (p, …) ↦ e | …      a function written out

p ::= c                   matches only itself
      x                   a placeholder: matches anything
      c(p, …)             matches data built by c, with contents matching p, …

h ::= (p) ↦ e             for the value the expression inside comes to
      (r(p, …), x) ↦ e    for a request, and a placeholder for the rest
```

TYPES

MAP

```
A ::= T(A, …)             a type, filled with types; T alone if it takes none
      X                   a placeholder for a type
      (A, …) → B ! E      a function type, and what it may ask;
                          ! E left out when it lists none
E ::= {F(A, …), …}        effects, filled; F alone if it takes none
```

STEPS

MAP

```
((p, …) ↦ e | …)(v, …)
    becomes  the body of the first case whose patterns match the inputs v, …,
             with what each placeholder matched in its place
n(v, …)
    becomes  the same, with the cases of n's definition
handle v with h | …
    becomes  the body of the first case for values that v matches,
             with what each placeholder matched in its place
handle e with h | …
    becomes  the body of the first case for r whose patterns match the inputs v, …,
             with what they matched in place, and the rest in place of x,
             when the next thing to work out in e is a request r(v, …),
             and no handler in e around that request has a case for r

the rest is (x) ↦ handle e′ with h | …, where e′ is e with x where the
  request was, and x is a placeholder used nowhere in e or in the handler's cases
```

WHERE STEPS HAPPEN

MAP

```
in an application, first in what is applied, until it is a value;
  then in its inputs, from left to right
in data, in its contents, from left to right
in a request, in its inputs, from left to right
in a handler, in the expression inside it
nothing happens inside a function until it is applied
```

HOW CARRYING OUT ENDS

MAP

```
in a value: the result
stuck: something is left to work out, no step applies, and it is not waiting
never: carrying out goes on forever
waiting: the next thing to work out is a request that no handler around it has
  a case for; when the outside answers, its answer goes where the request was,
  and carrying out goes on
```

NAMES

MAP

```
each type and each effect is declared once; each constructor belongs to one type,
  and each request to one effect; each defined name is defined once
a declaration or a definition can use any type or defined name, its own included,
  even one declared or defined after it
in a declaration, every name in a type is a declared type, or one of the
  declaration's own placeholders; those are all different, and none has the name
  of a type
a type or an effect is filled with as many types as it takes
in a pattern, a name is a constructor if it was declared as one,
  and otherwise a placeholder
the placeholders of one case are all different, and none has the name
  of a defined name or a request
a placeholder means something in the body of its own case,
  functions inside it included
in an expression, a name means the nearest placeholder of that name;
  with none, it is a constructor, a request or a defined name
the cases of one function all have the same number of patterns
a defined name written right before inputs is given as many as its function takes
a constructor is written with as many contents as it holds;
  one that holds nothing, without parentheses
a request is written with as many inputs as it is declared with
```

WHAT A TYPE MEANS

MAP

```
a value is of type T(A, …) when it is built by one of T's constructors,
  holding values of the types declared for its contents,
  with A, … in place of T's placeholders
a value is of type (A, …) → B ! E when, applied to values of types A, …,
  it never gets stuck, leaves no request unanswered but those of the effects
  in E, with inputs of the types they take, and, with each of those answered
  by a value of the type it is answered with, ends, if it ends, in a value
  of type B
a type that holds itself: a value is of it when following the definition,
  as far as it can be followed, never finds anything that does not fit
a type can hold itself in its contents, and in what a function gives back,
  but not in what a function takes

nothing in carrying out looks at types
```

THE SHAPE

	made by	used by
data	a constructor, holding values	a pattern, which takes it apart
function	cases, each with patterns and a body	applying it to inputs
effect	a request	a handler, which answers it

Six forms of expression for saying, two for doing, three forms of type. Everything else is a combination of them.